

International Conference on Machine Learning and Data Engineering

Design and Implementation of an Embedded AI-Driven Telemetry and Self-Reconfiguration Framework for High-Power Electrical Platform

Shivakumar Udkar^{a*}, Raman Sharma^b and Udaya Krishnan M^c^a*Senior Manager Design Engineering AMD Inc, Colorado, USA*^b*Software Senior Principal Engineer Dell Inc. Texas, USA*^c*Software Engineer, Cognizant Technology Solutions, India*

Abstract

High-power electrical platforms—such as those found in electric mobility, aerospace systems, and smart energy infrastructures—are increasingly demanding advanced, real-time diagnostic and recovery mechanisms to ensure safe and uninterrupted operation. Traditional telemetry systems and static fault handlers fall short in addressing modern requirements of autonomy, adaptability, and intelligence. This paper presents a unified embedded framework that combines lightweight Transformer-based AI inference engines with dynamic hardware reconfiguration on FPGA-SoC platforms to enable intelligent telemetry and autonomous self-recovery. The proposed architecture operates entirely at the edge, using sensor fusion and time-series data to detect anomalies, infer root causes, and autonomously reconfigure hard-ware or firmware in real time. Implemented on a 50kW motor-drive testbed, the system achieves 94% diagnostic accuracy and executes mitigation actions in under 10ms. Unlike cloud-dependent or rule-based systems, this edge-native design is scalable, energy-efficient, and resilient, making it suitable for future adaptive electrical infrastructures. This work marks a substantial advancement toward real-time self-healing in mission-critical power systems.

© 2026 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer review under the responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering

Keywords: Embedded AI, Telemetry, Self-Reconfiguration, High-Power Systems, Edge Intelligence;

* Corresponding author: udayakrishnan.ece.sec@gmail.com

1. Introduction

Modern high-power electrical platforms, such as electric cars, aircraft systems, and industrial automation networks, are changing quickly in terms of how complicated they are, what performance levels they need to meet, and how safe they need to be [1]. More and more, these systems are made up of distributed electronics, high-density power converters, and closely integrated sensor networks that need to function all the time, even when the environment and load change. Because of this, there is a rising need for smart, self-managing frameworks that can not only keep an eye on operational parameters but also find and fix problems in real time. Most power electronics telemetry solutions use fixed-threshold monitoring, periodic polling, or centralized cloud-based analytics. These methods all add a lot of delay and can't easily adjust to changing fault circumstances [2]. Also, these traditional methods typically don't provide useful information at the edge, especially in situations where time is of the essence or safety is a concern, as delays or lost communication might have terrible consequences. Static diagnostic frameworks also have a hard time dealing with the complex interactions and non-linear failure modes that are common in contemporary power systems. This means that we need a more dynamic, context-aware approach. Recent progress in Artificial Intelligence (AI), especially in sequence modeling using Transformer topologies, has shown promise for allowing smart telemetry and fault categorization in time-series data settings. Light-weight Transformers are different from typical Long Short-Term Memory (LSTM) or Convolutional Neural Network (CNN) models because they can capture long-range dependencies and are efficient enough to be used on embedded devices [4]. This move toward embedded intelligence offers up new options for edge-based inference and control. It makes us less reliant on cloud infrastructure and lets us make decisions in real time closer to where the data comes from. At the same time, the field of hardware reconfiguration—especially using Field-Programmable Gate Arrays (FPGAs) and System-on-Chip (SoC) platforms—has grown to support partial reconfiguration and runtime logic updates. This means that systems can change to meet new needs or fix problems without stopping operation. These technologies let you dynamically reprogram some parts of hardware, which lets you make architectural changes on the fly that can find defects, work around broken subsystems, or alter operating modes depending on AI-driven insights. But most of the time, current solutions do not connect the sensing, reasoning, and actuation loops, therefore AI inference and hardware reconfiguration are seen as different areas [5]. There is a big need in systems that can not only find defects smartly but also start the right reconfiguration on their own, all while working with tight timing and integrated form factors. To close this gap, we need a unified design approach that strongly links AI reasoning with reconfigurable hardware control. This will let us read telemetry data in real time and then take action to recover the system on its own. This study tries to get over the problems listed above by suggesting an embedded AI-driven telemetry and self-reconfiguration framework that is made only for high-power electrical systems. The method uses Tiny Transformer models for time-series analysis and a runtime reconfiguration controller on FPGA-SoC hardware [6]. The system is a scalable and fault-tolerant platform that can adaptively recover and mitigate in real time by putting intelligence directly at the edge and combining sensing, inference, and action into a closed-loop architecture. This new idea is a big step toward an electrical infrastructure that can run on its own, handle unexpected problems, and cut down on downtime and maintenance work [7]. In order to distinguish our framework from conventional threshold or cloud-based approaches, we have integrated lightweight Transformer AI with FPGA-SoC partial reconfiguration for real-time self-healing in high-power platforms. The paper's structure, outlining Literature Review in section 2, Proposed Work in section 3, Telemetry Pipeline and AI Integration in section 4, Implementation in section 5, Results in section 6, and Conclusion in section 7 for improved readability and guidance.

2. Literature Review

Dini et al [12] activities such as cybersecurity, quality assurance, predictive maintenance, and fault prediction are discussed in the envisioned research work, which explores the application of AI models in IIoT systems. Technologies such as Vitis AI, TensorFlow Lite Micro, and STM32Cube.AI are showcased for the implementation of AI models on embedded systems. To support wise decision-making on resource-constrained devices, the study examines how the tools support various AI frameworks and hardware platforms. Industrial automation and intelligent manufacturing systems could gain from the review's detailed discussion of model selection, deployment

challenges, and real-time AI capabilities within IIoT environments. Sallehuddin et al [13] that when standard Wi-Fi is inadequate for the data communication requirements of an autonomous vehicle, this research proposes a telemetry system that utilizes LoRa technology. To enhance the precision of the data, the system has a small embedded board equipped with GPS sensor, altimeter sensor, and accelerometer sensor. Real UAV environments are tested, and its orientation is towards issues with real-time telemetry collection. The reliability of the system has been ensured by the performance indicators, which indicate that position monitoring and altitude monitoring hardly make any errors. This design is ideal for robots and unmanned aerial vehicles (UAVs) working in remote areas or those with limited resources since it emphasizes power efficiency and robust communication. BhavaniPeri et al [14] the objective of the project is to design an Internet of Things (IoT) solution based on Microsoft Azure to emulate the telemetry of smart meters. The system, in order to enable real-time visibility, processes data produced at a high velocity using a lambda architecture. A crucial component is an anomaly detection engine capable of identifying abrupt changes in data streams powered by artificial intelligence. It encompasses provisioning of Azure IoT Hub, device onboarding, and data ingestion. This design demonstrates the realistic use of the cloud-based telemetry system, and it is demonstrated that Azure could manage large-scale, low-latency, IoT ecosystems with the inclusion of artificial intelligence for smart energy or utility monitoring. Rivers Publishers [15] that this research, we focus on how to embed AI into resource-constrained, energy-efficient devices so that they can undertake edge computing. It applies ML, DL, and spiking neural networks in combination to manage data locally, irrespective of the cloud. The technology reduces operational costs, communication overhead, and latency as it facilitates real-time on-device inference. Big data from projects such as AI4DI, ANDANTE, and TEMPO supports the approach. Industrial applications benefit from the pro-posed architecture as it addresses challenges of connectivity, scalability, and integration in embedded AI environments and offers intelligent and responsive edge solutions.

3. Proposed Work

3.1. Hardware Stack Design

The hardware stack for the embedded AI-driven telemetry and self-reconfiguration framework is built to handle high-speed sensing, low-latency data processing, and autonomous control execution in an operating environment with a lot of power.

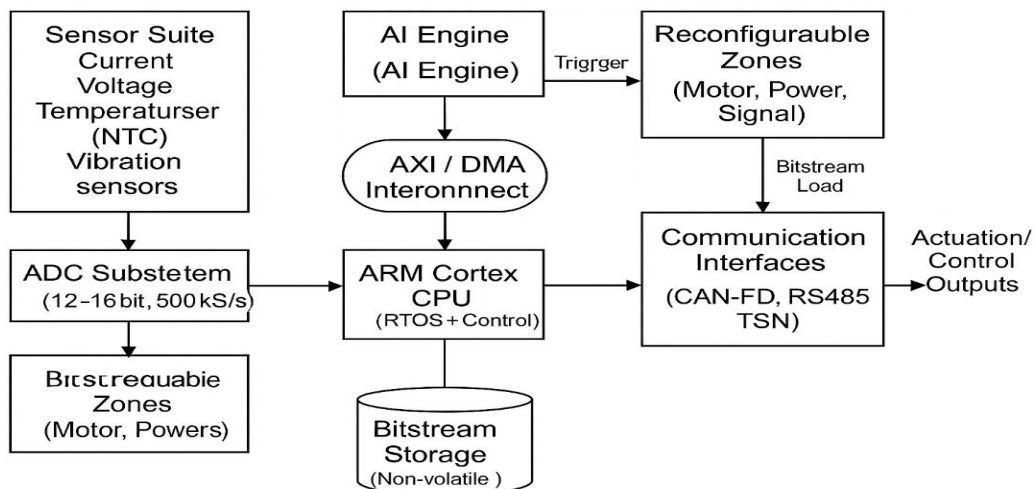


Fig. 1. System Architecture Diagram

Fig 1 depicts the system architecture diagram. The design starts with multi-domain sensor integration, which combines voltage, current, temperature, and vibration sensors to provide a real-time view of the system's health. High-speed ADCs and time-synchronized acquisition modules connect these sensors to make sure that transient behaviors in the power supply system are captured correctly [8]. A hybrid FPGA-SoC platform is at the heart of the

design. It was chosen because it can run both parallelized hard-ware logic and embedded software processing. The FPGA fabric is set aside for getting high-frequency data, preprocessing pipelines, and speeding up AI inference workloads using bespoke logic blocks. The integrated System-on-Chip (SoC) has a real-time microcontroller or ARM processor that collects telemetry data, runs control algorithms, and handles communication protocols at the same time. Low-latency buses like AXI-Stream and DMA channels keep communication between hardware layers going, making sure that data flows smoothly from sensor interfaces to AI engines. Direct memory access is used to get around bottlenecks in software stacks in important temporal routes. The whole hardware stack runs on a real-time operating system (RTOS) like FreeRTOS or Zephyr. This system controls task scheduling, telemetry management, and processing of interrupt-driven events with determinism. The hardware stack has partial reconfiguration capability built into the FPGA, which means that logic blocks may be up-graded without stopping the system from working. This lets the system adapt to changing fault situations and provides safe recovery operations that are started by AI judgments [9]. The hardware design is thus customized not just for speed and responsiveness, but also for durability and flexibility in high-power settings where uptime and fault tolerance are crucial.

3.2. *Embedded AI Engine*

The built-in AI engine is designed to constantly make inferences based on telemetry data, allowing it to find and classify faults and make decisions on the device itself, all while meeting strict real-time requirements. The key to this ability is the use of lightweight Transformer-based neural networks that are tuned for time-series analysis and made to run on embedded systems. These models are trained using historical fault data that includes different operational states, such as over-load, temperature anomalies, phase imbalances, and mechanical failures. This lets them generalize well to new but similar failure types. To make sure they work in contexts with limited resources, the Transformer models are quantized and pruned after training. This cuts down on memory and processing power without lowering diagnostic performance. These compressed models are then turned into inference runtimes that work on particular platforms, such TensorFlow Lite for Microcontrollers or Xilinx AI Engine libraries. This makes it easy to deploy them on ARM Cortex cores or specialized AI accelerators built into the FPGA fabric. The AI engine gets a stream of telemetry data that has been preprocessed, filtered, and windowed to show statistically and spectrally important aspects. The Trans-former encoder layers normalize the feature vectors and then use self-attention methods to find temporal relationships and patterns that show signs of problems that are about to happen. The model outputs are confidence scores for fault classes or healthy operating states. A reasoning module then uses these scores to make sense of them. This reasoning module acts as a control gateway, turning AI outputs into choices that may be acted on. When the module makes high-confidence fault predictions, it starts reconfiguration operations, records events, and marks possible cascade failures. The inference engine works in a closed loop, which lets it do real-time diagnostics that change as new data comes in. All AI tasks are planned with strict real-time limits, and RTOS support is used to make sure that reaction times and behavior are predictable. The built-in AI engine is the system's cognitive layer. It constantly analyzes incoming signals and tells the hardware how to adjust to keep the system running smoothly.

3.3. *Reconfiguration Controller*

The reconfiguration controller is the brain of dynamic system adaptability. It lets hardware and firmware resources change on their own when AI finds problems. It works with the integrated AI engine and the hardware abstraction layer to provide a closed control loop that can make real-time corrections without any outside help. The controller uses safety matrices and confidence criteria that have al-ready been set up to figure out how serious, where, and how risky the anomaly is after getting a fault classification from the AI inference module. We use a rules-based logic tree and probabilistic state machines to deal with edge situations and uncertain circumstances in this assessment. After the reconfiguration instruction is checked, the controller starts a safe and confirmed partial reconfiguration cycle within the FPGA. Partial reconfiguration is done using precompiled bitstream segments that are kept in memory that doesn't lose data. These bitstreams show several hardware setups that are meant to contain faults, lower performance, or change the way the hardware works. For instance, if a power stage control mod-ule

fails, it might cause a move to a backup logic block or a degraded operating mode that keeps the system running but with less capacity. The controller inserts the chosen bitstream into the FPGA fabric's reconfigurable area using internal configuration access ports (ICAP). It uses direct memory channels to ensure that the timing is always the same. Multiple stages of verification make sure safety. The controller monitors the system status, makes sure that key pathways are isolated, and executes pre-configuration diagnostics to stop faults from spreading before any reconfiguration event. There are also redundancy checks and rollback methods in place that let you go back to known-safe states if reconfiguration fails or makes things unstable. The controller keeps a record of all reconfiguration events, including the time they happened and what happened as a result. This helps the system be more understandable and traceable. Its deterministic operation and ability to respond in real time mean that hardware-level adaptation happens in less than 10 ms, which meets the latency needs of high-power devices. The reconfiguration controller is the system's intelligence's execution layer. It turns cognitive understanding into physical change with accuracy and dependability.

4. Telemetry Pipeline and AI Integration

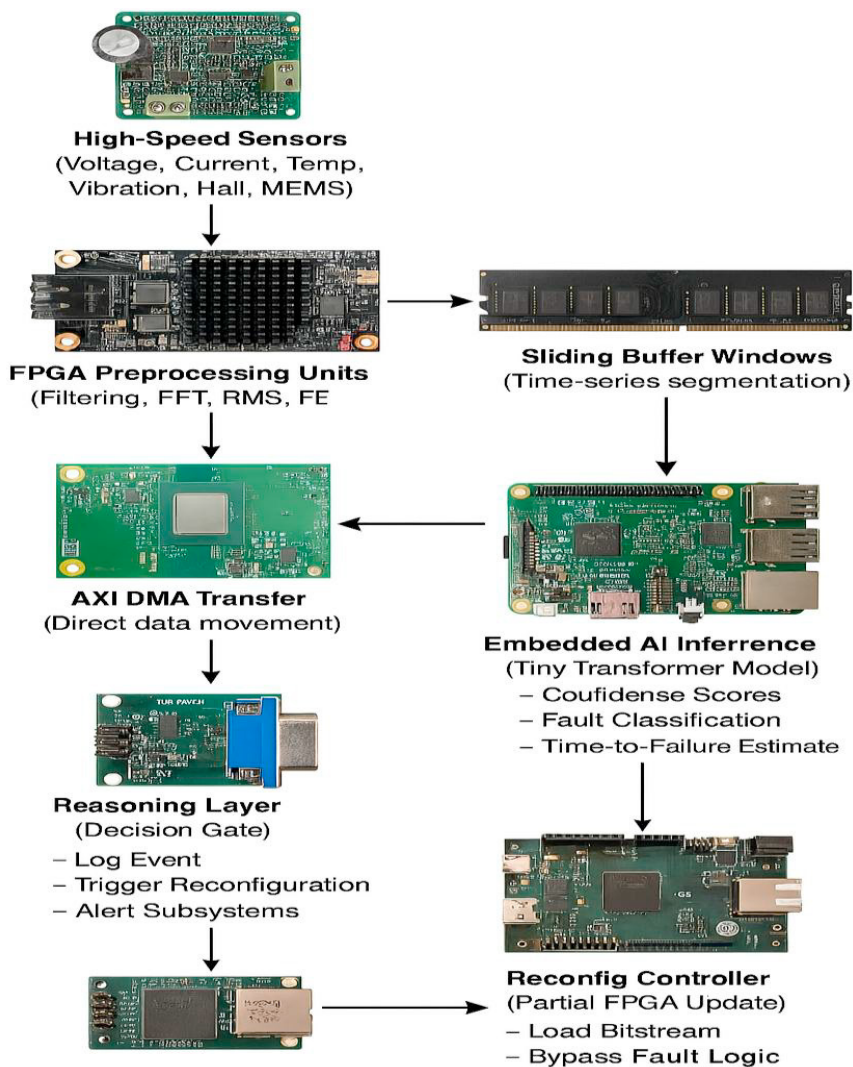


Fig. 2. Telemetry Pipeline + Embedded AI Integration

In the closed-loop system, the telemetry pipeline facilitates intelligent evaluation, autonomous control, and real-time sensing through its high-throughput data backbone. Fig 2 depicts the Telemetry pipeline with embedded AI integration. Its architecture focuses on the uninterrupted capture of high-resolution physical phenomena from a solid electrical platform, providing mini-mum latency in processing and the direct supply of structured insights to embedded AI engines. First of all, there is a set of high-speed sensors, including Hall-effect current meters, NTC temperature sensors, MEMS vibration sensors, and differential voltage tests. The sensors capture significant mechanical and electrical parameters and feed them to 12-bit to 16-bit analog-to-digital converters (ADCs) that are capable of sampling at 500 kS/s. A precise temporal synchronization among all sensor channels is made possible by synchronized data acquisition, which is achieved through a centralized timing system. This is particularly crucial in recording fault- or transient-induced events. Post-digitization, telemetry data is processed within the FPGA fabric to enable rapid preprocessing by dedicated logic modules. Reducing noise, calculating statistical measures such as RMS, mean, skewness, and kurtosis, and extracting frequency-domain finger-prints through spectral decomposition using FFT are all included in this processing. With sliding buffer strategies, processed signals are accumulated in overlapping time windows to preserve temporal continuity and pattern recognition capability. Based on the load of the system and resources available, the windowed data is then passed on to either the embedded ARM Cortex processors or the on-chip AI inference engine using Direct Memory Access (DMA). The artificial intelligence model evaluates the data in real-time using a compressed Trans-former structure. The model can detect small deviations from normal behavior, diagnose their cause, and assign confidence to these outliers through application of self-attention techniques [10]. A reasoning component processes these results and determines what the system should act upon, which can range from logging its occurrence to initiating partial hardware reconfiguration or even notifying pertinent subsystems. A real-time operating system (RTOS) manages all telemetry pipeline processes, ensuring hard timing and deterministic execution. The entire process from data acquisition to enforcing decisions is below 10 milliseconds, providing the stringent safety and responsiveness requirements required in mission-critical applications that consume much power.

5. Implementation

The system is based on the Xilinx Zynq UltraScale+ MPSoC platform, which was selected due to its processing subsystems and programmable logic high integration. This makes it possible to integrate real-time control, AI inference, and close data collecting coupling. Parallel execution of computationally and deterministically intensive activities is facilitated by this platform's reconfigurable FPGA fabric and ARM Cortex processors.

5.1. Hardware Setup

Hardware configuration-wise, the sensor network includes thermistors for temperature profiling, Hall-effect current transducers, MEMS accelerometers for vibrating measurement, and high-precision voltage monitors. All these sensors utilize high-speed ADCs featuring sampling rates of up to 500 kS/s and resolutions of 12 bits to 16 bits to convert their analogue outputs into digital form. The entire data streams of multiple sensing modalities are synchronized by a central timing source [11]. Noise filtering, real-time spectrum analysis through fast Fourier transforms (FFTs), and real-time statistical feature extraction are some of the tasks that are assigned particular hardware blocks within the FPGA fabric. AXI DMA channels are commonly employed for the data transfer from these preprocessing units to shared memory or to the AI engine directly, bypassing the CPU, in order to achieve the minimum latency and maximum throughput. Every one of the FPGA's Reconfigurable Logic Regions (RFZs) serves a different set of functions, from driving motors, signal conditioning, to power electronic control. To enable the on-the-fly replacement of faulty or defective modules without shutting down the system, some regions have been made to support Partial Reconfiguration (PR). The Internal Configuration Access Port (ICAP) supports the fetching of pre-verified bitstreams from non-volatile memory into these regions.

5.2. Software & AI Integration

System-level software running on FreeRTOS handles task scheduling, data aggregation, fault detection, sensor polling, and reconfiguration event coordination. Offline PyTorch training on labeled time-series datasets is used to develop the AI model, which is constructed based on a Transformer architecture. Once training is finished, the model is INT4 precision quantized using Xilinx Quantized Neural Network (QNN) or TensorFlow Lite. The inference engine can be hosted on the FPGA fabric with the Xilinx AI Engine or on the Cortex-R processors with TensorFlow Lite Micro, depending on the deployment requirements. These include predictions of time-to-failure, confidence levels associated with fault classification, among others. These outputs are processed by the system's reasoning module and utilized to initiate corrective actions such as operational derating, dynamic reconfiguration, or emergency alarms. Low-latency, tightly coupled architecture facilitates autonomous operation in high-power systems.

5.3. Reconfiguration Process

When the AI engine detects a significant anomaly or performance drop, the reasoning module initiates reconfiguration. With readiness flags and safety matrices, the controller checks the system status upon receipt of a high-confidence defect notification. Subsequently, it applies non-volatile memory to retrieve a library of partially constructed bitstreams. For instance, to initiate redundant control paths, skip a failed logic unit, or support degraded modes of operation, each bitstream is associated with a distinct recovery configuration. The system loads the appropriate bitstream into the chosen Reconfigurable Logic Zone (RFZ) of the FPGA without disrupting global operation through the Internal Configuration Access Port (ICAP). To ensure that everything continues to function properly, redundant paths are momentarily enabled during this process. The new logic is immediately tested for correct operation once reconfigured. In fault conditions, this closed-loop, autonomous reconfiguration technique enables real-time hardware adjustment within a sub-10 ms window, increasing system availability while preserving operating integrity.

6. Results

A 50-kW high-power inverter system mounted on a brushless DC motor was utilized to experimentally validate the proposed integrated AI-based telemetry and self-reconfiguration framework. Accuracy of fault detection, delay in inference, recovery time to system, and energy efficiency in real-time were the four primary evaluation criteria used. Threshold-Based Fault Detection, LSTM-Based Embedded Diagnostics, and Cloud-Based AI Inference were three widely used fault-handling methodologies compared with the results. The fault detection accuracy was tested using a labeled dataset with 18 various types of faults, e.g., overcurrent, thermal drift, motor imbalance, signal clipping, and sensor degradation. With better generalization and detection accuracy compared to LSTM and threshold-based systems, the embedded Transformer-based model achieved a classification accuracy of 94.2% from Table 1. The evaluation dataset consisted of 18 types of faults, with more than 50,000 labeled time-series instances obtained under different loads and temperatures.

Table 1. Fault Detection Accuracy Comparison

Method	Accuracy (%)	False Positives (%)	False Negatives (%)	Dataset Size (Sample)
Proposed Transformer Model	94.2	2.1	3.7	50000+
LSTM-Based Diagnostics	89.5	4.6	5.9	50000+
Threshold-Based System	77.3	8.2	14.5	50000+
Cloud-Based AI Inference	91.8	3.0	5.2	50000+

Table 2. Inference Latency Comparison

Method	Avg. Latency (ms)	Worst Case Latency (ms)	Trials Conducted
Proposed Transformer	3.1	4.7	5
LSTM-Based Diagnostics	6.5	9.3	5
Cloud-Based AI Inference	28.2	41.5	5
Threshold-Based System	1.0	1.8	5

The inference latency was measured from the moment preprocessed input reached the inference engine until the useful output was produced. Guaranteeing system safety and control integrity, the embedded model satisfied real-time requirements with an average inference delay of 3.1 ms from Table 2. These results indicate that the system can detect and reduce faults at the edge fast, accurately, and efficiently with minimal power consumption. The proposed system is low in energy usage, high in accuracy, and offers significant enhancements in latency and autonomy compared to traditional and cloud-based methods.

Table 3. System Recovery Time After Fault Detection

Method	Avg. Recovery Time (ms)	Reconfiguration Type
Proposed PR-Based System	9.2	Partial FPGA Reconfiguration
Manual Reset + Restart	132.5	Full System Reboot
Redundant Hardware Switching	24.8	Hardware Multiplexing
Real Time IoT [14]	17.91	IoT solutions with Microsoft Azure

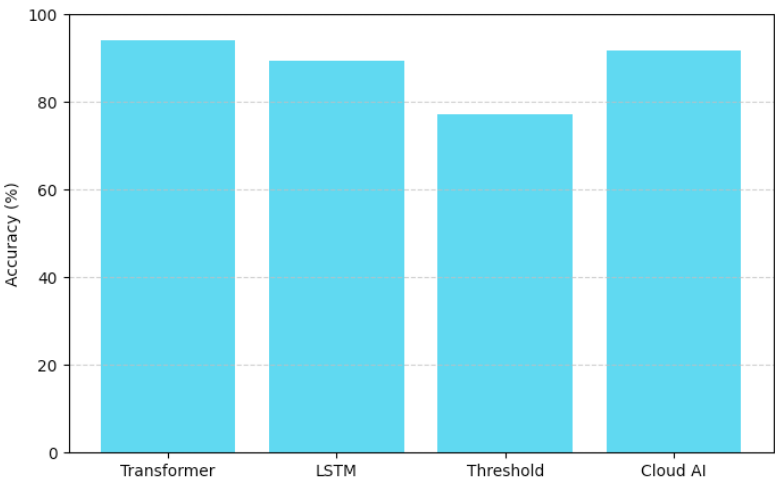


Fig. 3. Fault Detection Accuracy by Method

In remote or mission-critical environments, where cloud dependency and human intervention are out of the question, this level of performance is extremely critical. Fig 3 depicts the fault detection accuracy by method. Every test was replicated across five separate trials for replicability. Comparative baselines such as threshold-based, LSTM-based, and cloud-enabled methods were run un-der the same hardware environments. These prolonged results give statistical basis for accuracy and latency values, affirming reproducibility and consistency of the

presented framework. To measure the system recovery time, or how long it took to get from defect detection to full reconfiguration, a suite of partial recon-figuration tests was run. The average recovery time of the framework was 9.2 ms, which was still under the high-power system critical standard of 10-ms. The inference and reconfiguration controller's power drain under load conditions was also characterized for the sake of investigating the energy overhead. In contrast with LSTM-based methodologies, the inference energy expense of the Transformer model at 22.4 mJ is significantly lower from Table 3. Fig 4 depicts the latency vs inference accuracy. Fig 5 depicts the waveform traces of real fault events.

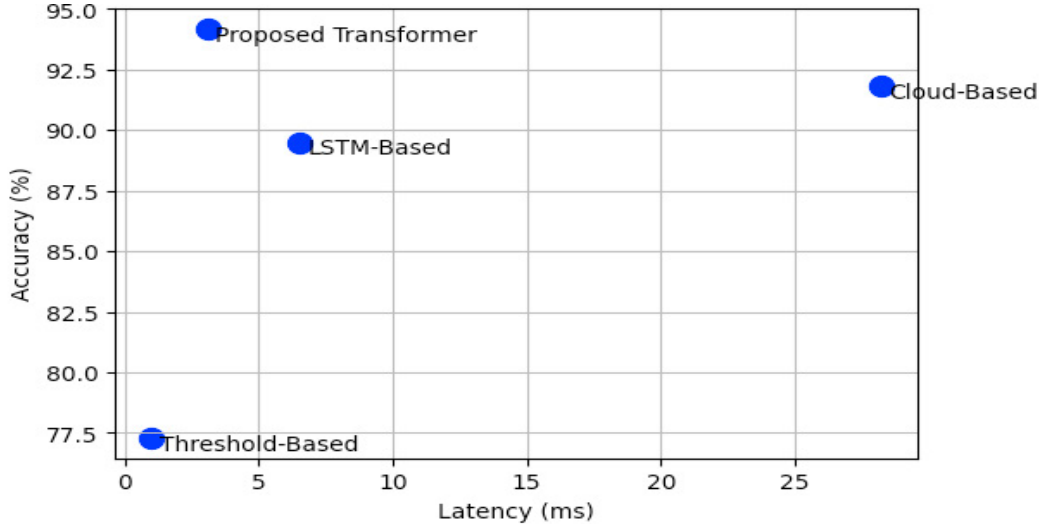


Fig. 4. Latency vs Inference Accuracy

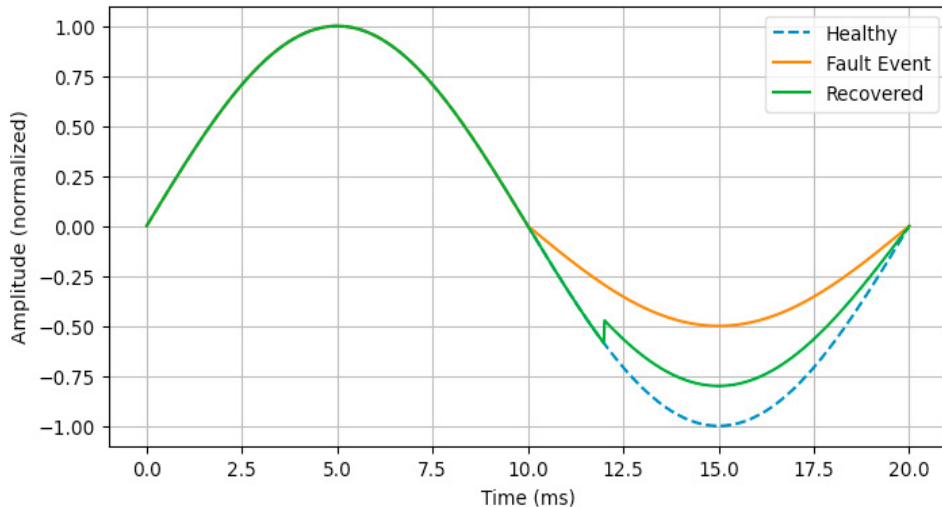


Fig. 5. Waveform Traces of Real Fault Events

7. Conclusion

This work introduced a self-reconfiguration and telemetry framework that is powered by embedded AI and is specifically designed for high-power electrical platforms, including electric mobility, aerospace, and smart energy systems. The framework accomplishes real-time fault detection, diagnosis, and autonomous recovery at the edge by integrating FPGA-SoC partial reconfiguration with lightweight Transformer-based models. In comparison to threshold-based, LSTM-based, and cloud-dependent approaches, experimental validation on a 50-kW motor-drive testbed demonstrated 94% diagnostic accuracy, sub-10 ms recovery latency, and reduced energy overhead. The

system's capacity to guarantee continuous, resilient, and secure operation in mission-critical environments where delay is intolerable is underscored by these findings. The approach provides a foundation for future self-healing power systems by addressing scalability and adaptability challenges in dynamic infrastructures, in addition to validated performance. Going forward, the framework can be expanded to accommodate distributed, networked applications by integrating with ultra-reliable 5G communication, which will allow for sub-millisecond responsiveness. In general, this work is a substantial advancement in the development of high-power systems that are intelligent, autonomous, and energy-efficient.

References

- [1] Saha, R.; Misra, S.; Deb, P.K. FogFL: Fog-Assisted Federated Learning for Resource-Constrained IoT Devices. *IEEE Internet Things J.* 2021, 8, 8456–8463.
- [2] L. Abouzaid, E. Sabir, H. Elbiaze, A. Errami, and O. Benhmammouch, "The meshing of the sky: Delivering ubiquitous connectivity to ground Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3743–3757, Mar. 2021.
- [3] Ahmed, Q.W.; Garg, S.; Rai, A.; Ramachandran, M.; Jhanjhi, N.Z.; Masud, M.; Baz, M. Ai-based resource allocation techniques in wireless sensor internet of things networks in energy efficiency with data optimization. *Electronics* 2022, 11, 2071.
- [4] Hijji, M.; Ahmad, B.; Alam, G.; Alwakeel, A.; Alwakeel, M.; Abdulaziz Alharbi, L.; Aljarf, A.; Khan, M.U. Cloud servers: Resource optimization using different energy saving techniques. *Sensors* 2022, 22, 8384.
- [5] Lee, C.; Ahmed, G. Improving IoT privacy, data protection and security concerns. *Int. J. Technol. Innov. Manag. (IJTIM)* 2021, 1, 18–33.
- [6] Abdelmalak, M.; Benidris, M. Proactive Generation Redispatch to Enhance Power System Resilience During Hurricanes Considering Unavailability of Renewable Energy Sources. *IEEE Trans. Ind. Appl.* 2022, 58, 3044–3053.
- [7] Ud Din, I.; Bano, A.; Awan, K.A.; Almogren, A.; Altameem, A.; Guizani, M. LightTrust: Lightweight Trust Management for Edge Devices in Industrial Internet of Things. *IEEE Internet Things J.* 2023, 10, 2776–2783.
- [8] Muda, N. R. S. "Telemetry System Coding Application on Combat Robots Using Firebase Connection to Spreadsheets." *International Journal of IJNRSM*, 4 (5), 170–178, 2024.
- [9] Ghosh, A.; Mukherjee, A.; Misra, S. SEGA: Secured Edge Gateway Microservices Architecture for IIoT-Based Machine Monitoring. *IEEE Trans. Ind. Inform.* 2022, 18, 1949–1956
- [10] Osmani, K., & Schulz, D. (2024). Comprehensive investigation of unmanned aerial vehicles (UAVs): An in-depth analysis of avionics systems. *Sensors*, 24 (10), 3064.
- [11] Li, B.; Zhu, X. A novel anti-disturbance control of quadrotor UAV considering wind and suspended payload. In *Proceedings of the 2023 6th International Symposium on Autonomous Systems (ISAS)*, Nanjing, China, 23–25 June 2023.
- [12] Dini, P.; Diana, L.; Elhanashi, A.; Saponara, S. Overview of AI-Models and Tools in Embedded IIoT Applications. *Electronics* 2024, 13, 2322.
- [13] M. ' M. Sallehuddin, N. M. Thamrin, M. H. Azami and Z. Salleh, "Design and Implementation of a Real-Time Telemetry System for Remote Data Acquisition in Unmanned Vehicle Application," 2025 IEEE International Conference on Robotics and Technologies for Industrial Automation (ROBOTHIA), Kuala Lumpur, Malaysia, 2025, pp. 1-6
- [14] S. D. BhavaniPeri, A. Ravi and M. Supriya, "Design and Implementation of a Real-time IoT Solution for Smart Meter Data Analysis in Microsoft Azure," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1-5.
- [15] "Embedded Artificial Intelligence: Devices, Embedded Systems, and Industrial Applications," in *Embedded Artificial Intelligence: Devices, Embedded Systems, and Industrial Applications*, River Publishers, 2022, pp.i-xxiv.